

Introduction

Every so often a LeetCode problem shows up that looks like a simple digit-manipulation exercise until you notice the constraints, and by then it's too late to save your naive solution. LeetCode 3348, Smallest Divisible Digit Product II, is exactly that kind of problem. On the surface it asks something innocent: find the smallest number greater than or equal to a given one, with no zero digits, whose digits multiply to something divisible by a target value `t`. But the number can be up to 200,000 digits long, which means "just check the next number, and the next" is not a real algorithm — it's a way to time out with confidence.

This problem is worth sitting with because it teaches a pattern that shows up constantly in interviews: when brute force incrementing is off the table, you look for a hard ceiling on what a fixed-size structure can produce. That ceiling tells you exactly when the current size is no longer enough and you have to commit to something bigger. Once you internalize that idea, a whole category of "smallest number satisfying X" problems stops feeling mysterious.

Problem Overview

You're given a number `num`, represented as a string of digits, and an integer `t`. You need to find the smallest integer that is:

1. Greater than or equal to `num`,
2. Made up entirely of nonzero digits (1 through 9, no zeros allowed),
3. Has a digit product divisible by `t`.

If no such number exists, return `-1`.

The zero-free constraint isn't decoration — it closes a loophole. If zero digits were allowed, the digit product would just be zero, and zero is divisible by everything, which would make the problem trivial and wrong.

Example

Take `num = "14"` and `t = 6`.

- `14` → digits multiply to $1 \times 4 = 4$. Not divisible by 6.

- $15 \rightarrow 1 \times 5 = 5$. Not divisible by 6.
- $16 \rightarrow 1 \times 6 = 6$. Divisible by 6. 

Answer: 16 .

That's the brute-force intuition working correctly on a tiny input. The entire rest of this article is about getting the same answer without walking through 15 first.

Intuition

The naive approach is obvious: start at `num`, check the digit product, increment, repeat. It's correct, and completely impractical once `num` has hundreds of thousands of digits — checking one candidate at a time could mean traversing an astronomical number of integers.

The way out is to stop thinking about numbers as things you check, and start thinking about them as things you build. Every digit is between 1 and 9, so a number of fixed length has a hard ceiling on how large a digit product it can produce. More importantly, digits only ever contribute prime factors of 2, 3, 5, or 7 — a digit can never contain a prime factor like 11 or 13, no matter how you choose it. Call this the **Nine Cap**, since 9 (which is 3^2) is the single most factor-density any one digit slot can ever hold.

This gives you an immediate feasibility check: strip every factor of 2, 3, 5, and 7 out of `t`. Whatever remains — if it isn't 1 — is a prime that no digit, and therefore no number, can ever supply. That's an instant `-1`.

If `t` only needs more 2s, 3s, 5s, or 7s than the digits can currently provide, the fix is to add or adjust digits, not to search for a whole new number. So the algorithm becomes: keep as much of `num`'s prefix as possible, and the moment the remaining digits can't cover what's needed, bump that digit to the smallest value that still leaves the requirement coverable, then fill everything after it with the smallest digits that finish the job.

Brute Force Solution

Idea: Start at `num`, and for each candidate, compute the digit product and check divisibility by `t`. If it's zero-free and divisible, return it. Otherwise move to the next integer.

Algorithm: 1. Set `candidate = num`. 2. While `candidate` has no zero digit and `product(candidate) % t != 0`: increment `candidate`. 3. Return `candidate`.

Advantages: Trivial to implement and easy to reason about correctness.

Disadvantages: Completely impractical for large inputs. The gap between consecutive valid numbers can be enormous, and with `num` up to 200,000 digits, incrementing one at a time never finishes within any reasonable time limit.

Complexity: Time is effectively unbounded in the worst case (potentially exponential relative to digit count); space is $O(L)$ for holding the digits, where L is the length of `num`.

Optimal Solution

The optimal approach factors `t` down into a small set of required prime counts, then constructs the answer left to right.

Step 1 — Factor and bail early. Divide `t` by 2, 3, 5, and 7 repeatedly, tracking how many of each you removed. If anything other than 1 remains, no digit can ever contribute that prime factor, so return `-1` immediately. This is the Nine Cap in action: it turns an impossible target into an instant rejection instead of a doomed search.

Step 2 — Try to keep `num` as-is. If `num`'s own digit product (after the zero check) already satisfies `t`, you're done — it's the smallest valid answer by definition, since it equals the input.

Step 3 — Find the first digit that must change. Walk `num` left to right. For each position, check whether keeping the current digit still leaves the remaining suffix able to cover whatever factors are still missing, using the remaining slots. The moment a digit can't be kept, that's your pivot point.

Step 4 — Bump and rebuild. At the pivot digit, choose the smallest digit value greater than the original that still leaves the remaining requirement coverable by the digits after it. Then fill every position after the pivot with the smallest digit, at each position, that still leaves the remaining requirement coverable in the remaining slots.

Step 5 — Grow the length if needed. If no digit at the current length can be bumped successfully, the length itself is insufficient (the Nine Cap for that length is too low). Increase the length by one and repeat the construction, since a zero-free number one digit longer is still smaller than skipping ahead further.

A key subproblem inside steps 3 and 4 is: "given `n` remaining digit slots, and a required count of 2s and 3s (5s and 7s are handled separately since each costs exactly one digit), what's the minimum number of digits needed to cover that requirement?" Digits 5 and 7 each supply exactly one factor of their own prime, so they cost one digit each. Twos and threes are more interesting because digit 6 supplies one factor of each simultaneously — using a single 6 can be cheaper than spending two separate digits. This subproblem is small (the needed counts are always bounded, since anything

beyond a handful of extra factors can just use 8s and 9s) and is solved once and cached, since it recurs constantly during the digit-by-digit greedy check.

Digit Factors contributed

1	none
2	one 2
3	one 3
4	two 2s
5	one 5
6	one 2, one 3
7	one 7
8	three 2s
9	two 3s

Python Code

```
from functools import lru_cache

def min_digits_needed(need_two: int, need_three: int) -> int:
    """Minimum digit count to supply at least need_two factors of 2
    and need_three factors of 3, using digits 1-9."""
    if need_two <= 0 and need_three <= 0:
        return 0

    @lru_cache(maxsize=None)
    def solve(two: int, three: int) -> int:
        if two <= 0 and three <= 0:
            return 0
        best = float("inf")
        # Try each digit's contribution, take the largest useful cut per digit.
        for two_cut, three_cut in ((3, 0), (2, 1), (1, 1), (0, 2)):
            rem_two = max(0, two - two_cut)
            rem_three = max(0, three - three_cut)
            if rem_two == two and rem_three == three:
                continue
            best = min(best, 1 + solve(rem_two, rem_three))
        return best

    return solve(need_two, need_three)

def factor_requirement(t: int):
    """Strip 2, 3, 5, 7 out of t. Returns (need_two, need_three, need_five,
    need_seven) or None if t has a leftover prime factor > 7."""
    need = {2: 0, 3: 0, 5: 0, 7: 0}
    for p in (2, 3, 5, 7):
```

```

        while t % p == 0:
            t //= p
            need[p] += 1
    if t != 1:
        return None
    return need[2], need[3], need[5], need[7]

def digit_product_ok(digits, need_two, need_three, need_five, need_seven) -> bool:
    two = three = five = seven = 0
    for ch in digits:
        d = int(ch)
        if d == 0:
            return False
        n = d
        while n % 2 == 0:
            n //= 2
            two += 1
        while n % 3 == 0:
            n //= 3
            three += 1
        while n % 5 == 0:
            n //= 5
            five += 1
        while n % 7 == 0:
            n //= 7
            seven += 1
    return two >= need_two and three >= need_three and five >= need_five and seven >= need_seven

def feasible_suffix(slots: int, need_two: int, need_three: int, need_five: int, need_seven: int) ->
    """Can `slots` remaining digits cover the remaining requirement?"""
    fives_sevens = need_five + need_seven
    if fives_sevens > slots:
        return False
    leftover_slots = slots - fives_sevens
    return min_digits_needed(need_two, need_three) <= leftover_slots

def smallest_valid_at_length(digits, need_two, need_three, need_five, need_seven):
    n = len(digits)
    digits = list(digits)

    def remaining_after(idx_digit_pairs):
        two = three = five = seven = need_two, need_three, need_five, need_seven
        return two, three, five, seven

    # Try keeping the prefix as long as possible.
    two, three, five, seven = need_two, need_three, need_five, need_seven
    covered = [0] * (n + 1) # placeholder not needed; compute directly below

    # Compute suffix feasibility by trying to keep each prefix, bump at first failure.
    for i in range(n):

```

```

d = int(digits[i])
if d == 0:
    d = 9 # will be forced to bump below since original had a zero
# requirement after using digits[0..i] as original (or bumped) digit
# First check: can we keep original digit here?
orig_digit = int(digits[i])
if orig_digit != 0:
    t2, t3, t5, t7 = reduce_need(need_two, need_three, need_five, need_seven, orig_digit)
    if feasible_suffix(n - i - 1, t2, t3, t5, t7):
        need_two, need_three, need_five, need_seven = t2, t3, t5, t7
        continue

# Must bump this digit to the smallest value > current that still works.
start = orig_digit + 1 if orig_digit != 0 else 1
for cand in range(start, 10):
    t2, t3, t5, t7 = reduce_need(need_two, need_three, need_five, need_seven, cand)
    if feasible_suffix(n - i - 1, t2, t3, t5, t7):
        digits[i] = str(cand)
        need_two, need_three, need_five, need_seven = t2, t3, t5, t7
        # Fill every slot after i greedily, smallest digit first.
        for j in range(i + 1, n):
            for fill in range(1, 10):
                f2, f3, f5, f7 = reduce_need(need_two, need_three, need_five, need_seven, f)
                if feasible_suffix(n - j - 1, f2, f3, f5, f7):
                    digits[j] = str(fill)
                    need_two, need_three, need_five, need_seven = f2, f3, f5, f7
                    break
            return "".join(digits)
        return None # no digit at position i works; length must grow

return "".join(digits) # original already worked

def reduce_need(need_two, need_three, need_five, need_seven, digit):
    factors = {
        1: (0, 0, 0, 0), 2: (1, 0, 0, 0), 3: (0, 1, 0, 0), 4: (2, 0, 0, 0),
        5: (0, 0, 1, 0), 6: (1, 1, 0, 0), 7: (0, 0, 0, 1), 8: (3, 0, 0, 0),
        9: (0, 2, 0, 0),
    }
    t2, t3, t5, t7 = factors[digit]
    return (max(0, need_two - t2), max(0, need_three - t3),
            max(0, need_five - t5), max(0, need_seven - t7))

def smallest_divisible_digit_product(num: str, t: int):
    req = factor_requirement(t)
    if req is None:
        return -1
    need_two, need_three, need_five, need_seven = req

    if digit_product_ok(num, need_two, need_three, need_five, need_seven):
        return num

```

```

length = len(num)
while True:
    if length == len(num):
        candidate_digits = list(num)
    else:
        candidate_digits = ["1"] + ["1"] * (length - 1)

    if not feasible_suffix(length, need_two, need_three, need_five, need_seven):
        length += 1
        continue

    result = smallest_valid_at_length(
        candidate_digits, need_two, need_three, need_five, need_seven
    )
    if result is not None:
        return result
    length += 1

```

Code Walkthrough

- `factor_requirement` strips 2, 3, 5, 7 out of `t`. Anything left over means `t` has a prime factor no digit can ever supply, so we return `None` and the caller returns `-1`.
- `min_digits_needed` (cached with `lru_cache`) answers: given a required count of extra 2s and 3s, what's the fewest digits needed to supply them, considering that digit 6 covers one of each at once? This is memoized because it's called repeatedly during the greedy fill.
- `feasible_suffix` checks whether a given number of remaining digit slots can still satisfy the outstanding requirement — 5s and 7s each cost exactly one dedicated digit, and whatever's left over must fit the cached 2/3 answer.
- `reduce_need` computes what requirement remains after "spending" a specific digit.
- `smallest_valid_at_length` walks the digits left to right, keeping the original digit wherever doing so still leaves the suffix feasible, and the moment it can't, bumps that digit up to the smallest workable value and greedily fills everything after it with the smallest digit that keeps the rest feasible.
- `smallest_divisible_digit_product` orchestrates everything: factor `t`, check if `num` already works, and if not, try to construct an answer at `num`'s length; if that's infeasible, grow the length until it is.

Dry Run

Take `num = "1234"`, `t = 256`. Since $256 = 2^8$, we need eight factors of 2 and nothing else.

- Original digits 1, 2, 3, 4 supply factors of 2 from 2 (one) and 4 (two) — three total. Not enough, and the product also includes odd digits, so `num` itself fails immediately.
- Walk left to right: digit 1 at position 0 supplies nothing, but check feasibility of the remaining three slots covering eight 2s — with three slots left, the max is $3 \times 3 = 9$ (using three 8s), so it's feasible. Keep 1.
- Digit 2 at position 1: keeping it means 2 slots remain needing $8 - 1 = 7$ more. Two slots max out at $3 + 3 = 6$. Not feasible. Bump: try 3 (no help, still need 8, infeasible), try 4 (two factors of 2, need 6 more, two slots max 6 — feasible!). Set digit to 4.
- Now two slots remain needing six factors of 2. Fill greedily: try 1 (no help, need 6, one slot left, max 3 — infeasible), ... try 8 (three factors of 2, need 3, one slot left, max 3 — feasible). Set to 8.
- Last slot needs 3 factors of 2: try 1 ... 7, none give exactly enough except 8 (three factors). Set to 8.

Result: 1, 4, 8, 8 → **1488**, matching the trace from the video.

Complexity Analysis

Time: Constructing the answer processes each digit position once, and at each position tries at most 9 candidate digits, each requiring an $O(1)$ lookup into the cached minimum-digits table (the cache itself has a small, bounded number of distinct (two, three) states since required counts are capped by what a handful of digits can supply). This gives roughly $O(L)$ where L is the length of `num`, compared to the brute-force approach's effectively exponential blowup (up to 9^L candidates in the worst case).

Space: $O(L)$ for the digit array, plus a small constant-size cache for the 2/3 subproblem.

The correctness of the time bound comes directly from the Nine Cap: once you know a fixed-size cache answers "how many digits does this leftover requirement need," every per-position decision becomes $O(1)$ amortized instead of a fresh search.

Alternative Solutions

A more general digit dynamic programming formulation exists, tracking tight/loose prefix state alongside remaining factor requirements as a full DP over positions. It produces the same answer and can be extended more easily to variants of this problem, but it carries more state and more code for no gain here. The greedy-plus-cache approach is simpler to write and to reason about, so it's the better choice unless the problem changes shape.

Edge Cases

- **Empty or single-digit `num`** : the algorithm still applies; a length of 1 just means very tight feasibility checks.
- **`t = 1`** : every zero-free number works; `num` itself is the answer immediately (as long as it has no zero digits).
- **`t` has a prime factor above 7** (like 11, 13, or any larger prime): return `-1` immediately — no length ever fixes this.
- **`num` contains a zero digit**: it fails the zero-free check outright, so the construction always has to move past it, even if the digit product would otherwise work.
- **Very large `need_two` / `need_three` relative to length**: feasibility check correctly rejects and forces the length to grow, as in the $99 \rightarrow 188$ example.

Common Mistakes

1. **Forgetting the zero-free rule.** A zero digit makes the product zero, and zero is divisible by everything — sloppy code can accept an invalid answer that technically "passes" the divisibility check.
2. **Falling back to brute-force incrementing** on the full-size input, which times out given up to 200,000 digits.
3. **Not handling the leftover prime factor case.** Skipping the check for primes above 7 in `t` leads to infinite loops or wrong answers, since no digit sequence can ever satisfy such a requirement.
4. **Assuming keeping the original digit is always possible.** Skipping the feasibility check on the suffix and just keeping `num`'s digits as-is can silently produce a number smaller than required, which fails divisibility.
5. **Ignoring digit 6's dual contribution.** Treating 2s and 3s as fully independent costs (one digit per unit) instead of recognizing that a single 6 covers both wastes digits and can produce an answer that's larger than necessary.

Interview Questions

- Why can't we brute-force this given the input size, and what's the actual growth rate of the search space?
- Why is 9 the effective cap for a single digit's "factor density," and how does that bound the algorithm's correctness?
- How would you extend this if digits could range over a different base (e.g., hexadecimal digits)?

- How do you handle ties when two digit choices produce equally feasible suffixes?
- Can you convert this greedy-plus-cache method into a full digit DP, and what extra state would you need?

Similar Problems

- **LeetCode 402: Remove K Digits** — another "smallest number under a constraint" problem solved by greedy digit-by-digit decisions rather than brute force.
- **LeetCode 738: Monotone Increasing Digits** — builds the largest valid number below a bound by adjusting digits from the point of first violation, a close cousin of the bump-and-rebuild step here.
- **LeetCode 670: Maximum Swap** — teaches the same "find the pivot position, then fix everything after it" mindset in a smaller, more approachable form.
- **LeetCode 179: Largest Number** — different objective, but the same family of "construct the answer instead of enumerating candidates" problems.

Key Takeaways

When a problem asks for the smallest (or largest) thing satisfying a numeric condition, and the search space is too big to enumerate, look for a hard ceiling on what a fixed-size structure can produce — here, that's the Nine Cap. That ceiling tells you exactly when the current size can't work, at which point you grow it deliberately instead of searching blindly. Building the answer left to right, keeping as much of the original as possible and only changing what has to change, turns an intractable search into a linear construction.

Watch the Video

If you want to see this traced out live, digit by digit, with the two quiz rounds worked through in real time, watch the full breakdown here: {LINK}

About the Series

This is part of Fun with Learning Technology, a series that takes a genuinely hard problem each day and breaks it down until the solution feels obvious rather than clever. No skipped steps, no hand-waving — just the reasoning that gets you from "this looks impossible" to "of course."

Call To Action

If this helped the problem click, the like button on the video is the easiest way to say so. Subscribe on YouTube for a new daily problem, and subscribe to the Substack if you want these written up the same way you just read them. Drop a comment with your own solution or a question about any step — and share this with anyone prepping for interviews who's stuck on digit-construction problems.

Quick Review

Pattern for 'smallest number with digit product divisible by t'?

Suffix-greedy digit search: fix the shortest changeable suffix, brute-force digits 1-9 per position, when a factorization target reduces to needing digits capped at 9.

Time complexity of the suffix + digit-cap search?

Roughly $O(n * 9^k)$ where k is a small fixed suffix window ($\sim 7-8$), since factors of t only need a few digits beyond a leading '1' run.

Space complexity of this approach?

$O(n)$ for the output digit array plus $O(1)$ extra for tracking remaining factors of 2, 3, 5, 7 (t 's prime pieces).

What's the trickiest edge case here?

t has a prime factor >9 (e.g. 7 needs a 7, but 11+ is impossible) — no digit 1-9 can supply it, so answer is -1.

Nine Cap trick vs brute-force digit incrementing?

Brute force increments the whole number one by one (times out at huge n); the Nine Cap fixes leading 1s and only searches a small suffix window.

Interview follow-up: why prefer suffix search over full DFS on all n digits?

Because leading digits should stay minimal (1s); only the last few digits need to hit the needed factors, so search space shrinks from 9^n to 9^k .

Smallest Divisible Digit Product II — free Python cheat sheet